Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 8B

Disciplina: Tehnici de programare

Tema: ”Unealta stricată, reparată cu git”

A efectuat : st. gr.SI – 261, Vizitiu Laura

A verificat : asistent universitar, Dumitrașcu Marius

Chișinău – 2026

Sarcina lucrării

exercitiu-git > depanare > inventar.py > main

```python
# inventar.py - rezumatul tintelor dintr-un fisier JSON
# Unealta a fost scrisa in graba si nu mai merge. Reparati-o.
# Rulare:  python inventar.py tinte.json --prag 1000
import argparse
import json


def incarca(cale):
    """Citeste lista de tinte din fisierul JSON."""
    with open(cale) as f:
        return json.load(f)


def port_valid(text):
    """Intoarce portul ca int, sau None daca nu e un numar."""
    try:
        return int(text)
    except ValueError:
        return None


def pe_servicii(tinte):
    """Dict: serviciu -> lista de porturi."""
    grupe = {}
    for t in tinte:
        serviciu = t.get("serviciu", "necunoscut")
        port = port_valid(t["port"])
        if port is None:
            print("sar peste", t["gazda"], ": port invalid", t["port"])
            continue
        grupe.setdefault(serviciu, [])
        grupe[serviciu].append(port)
    return grupe

```

Imaginea 1.1 Codul reparat

```python
def peste_prag(tinte, prag):
    """Lista gazdelor cu un port mai mare decat pragul."""
    rezultat = []
    for t in tinte:
        port = port_valid(t["port"])
        if port is not None and port > prag:
            rezultat.append(t["gazda"])
    return rezultat


def main():
    p = argparse.ArgumentParser()
    p.add_argument("fisier")
    p.add_argument("--prag",type=int, default=1024)
    a = p.parse_args()

    tinte = incarca(a.fisier)
    print("Tinte citite: " , len(tinte))

    grupe = pe_servicii(tinte)
    for serviciu, porturi in grupe.items():
        print(f"{serviciu}: {porturi}")

    mari = peste_prag(tinte, a.prag)
    print(f"Peste {a.prag}: {mari}")

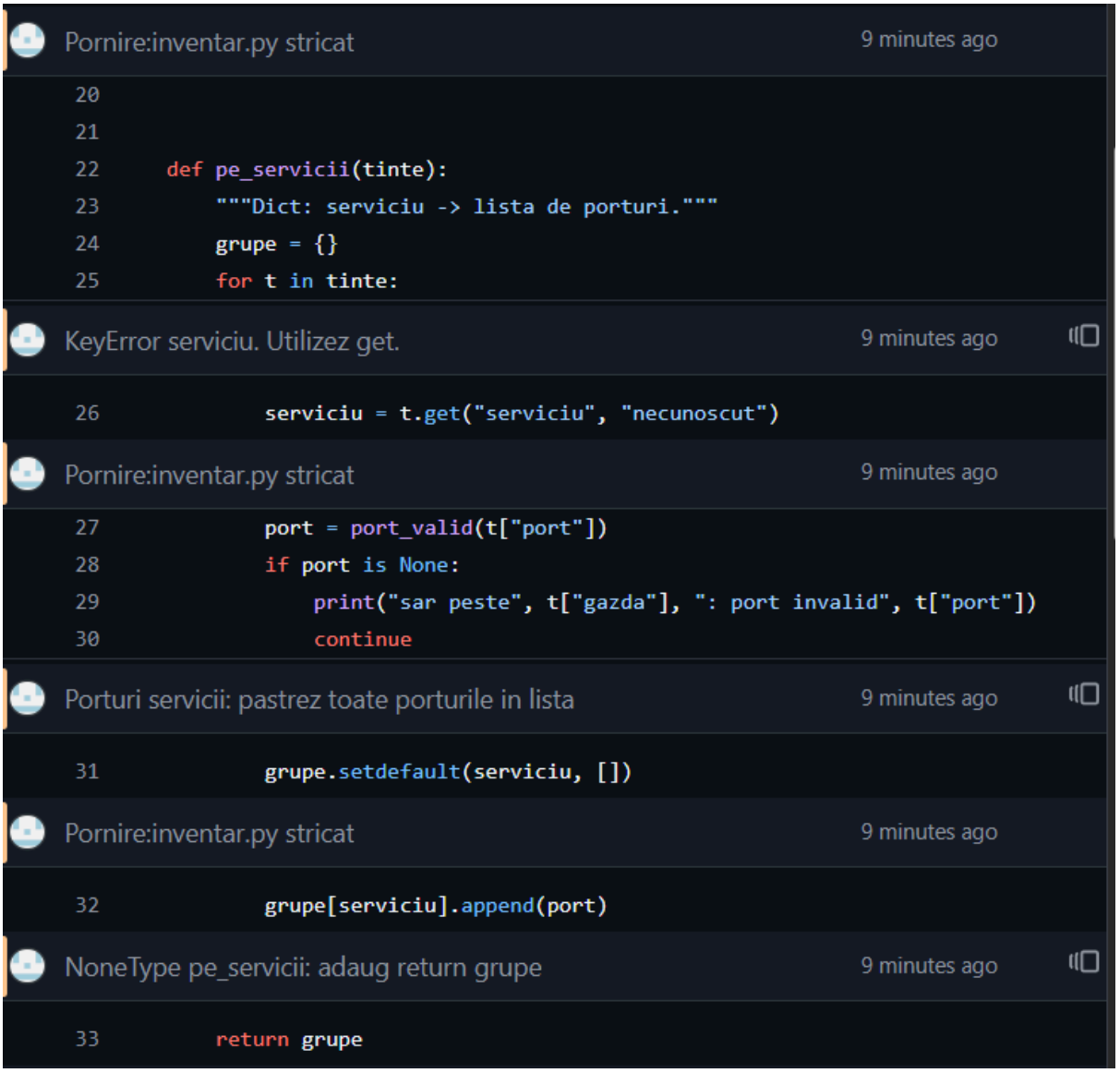
    with open("raport_inventar.json", "w") as f:
        json.dump({"servicii": grupe, "peste_prag": mari}, f, indent=2)
    print("Raport salvat in raport_inventar.json")


if __name__ == "__main__":
    main()
```

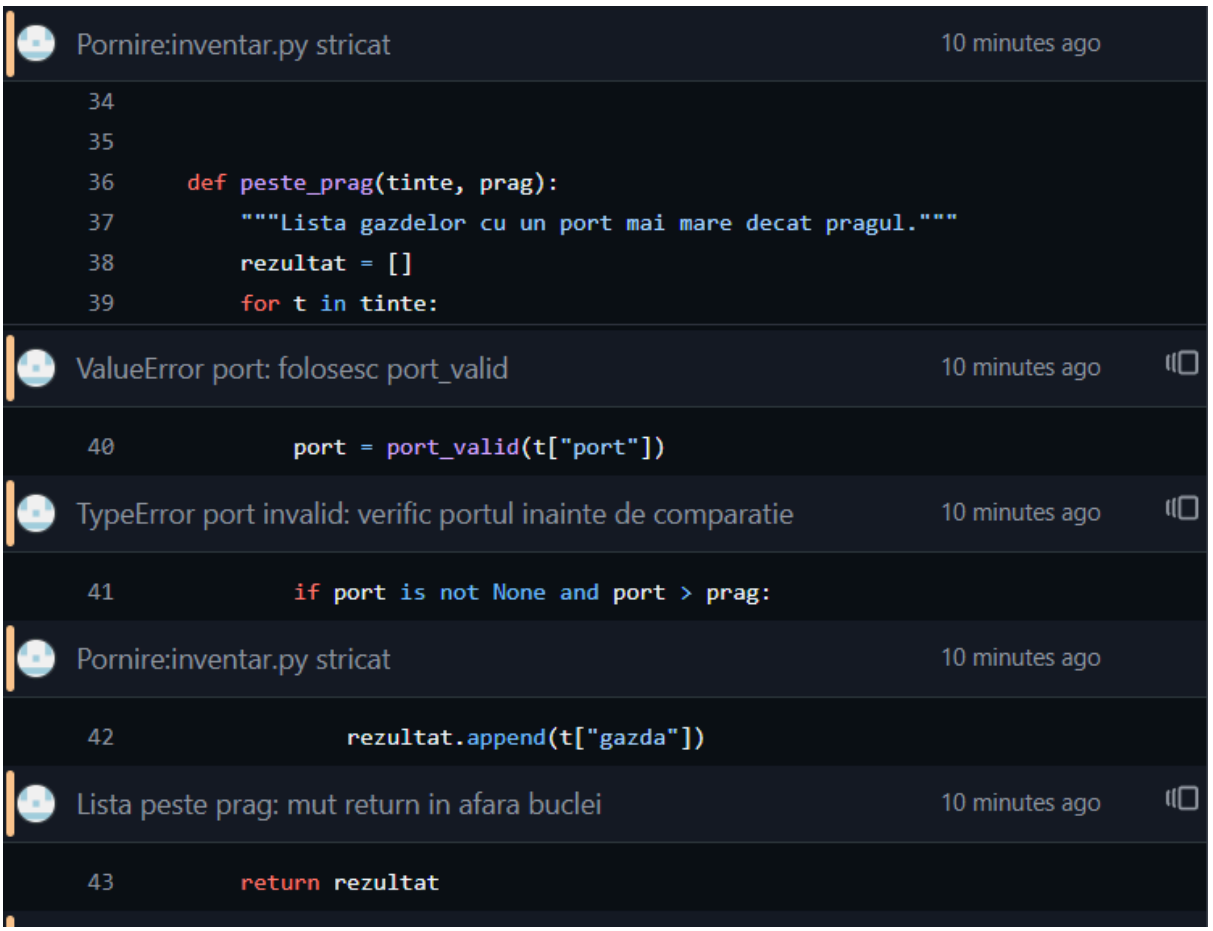
imaginea 1.2 Codul reparat



Imaginea 2.1 Modificări



Imaginea 2.2 Modificări



Imaginea 2.3 Modificări



Imaginea 2.4 Modificări

Concluzie

În urma acestei lucrări, am învățat să identific și să repar erorile dintr-un program Python folosind Git. Am corectat problemele legate de accesarea datelor, gruparea porturilor, conversia valorilor și verificarea porturilor invalide. De asemenea, am învățat să folosesc comenzile git add, git commit, git pull și git push pentru a salva și sincroniza modificările cu depozitul GitHub. Am verificat funcționarea programului prin executarea comenzilor și generarea raportului în format JSON. Astfel, am înțeles cât de important este să repar erorile treptat, să salvez fiecare modificare printr-un commit și să verific rezultatul final.